

1 Objetivo do Trabalho

O objetivo deste trabalho é desenvolver agentes inteligentes em Prolog para um jogo de perseguição no estilo “Detetive vs Ladrão”. Cada grupo deverá implementar um agente autônomo capaz de tomar decisões ao longo da partida com base nas informações disponíveis.

O jogo ocorre em um conjunto de cidades conectadas formando um grafo. Há dois tipos de agentes:

- um **ladrão**, cujo objetivo é roubar um tesouro específico;
- um **detetive**, cujo objetivo é impedir o roubo ou capturar o ladrão.

Cada agente interage com uma máquina de jogo por meio de funções específicas. O agente pode apenas informar à máquina de jogo a ação a ser realizada e não pode manipular diretamente o estado do jogo ou seu próprio estado.

O jogo é baseado em turnos. Em cada turno:

1. o ladrão executa uma ação;
2. o estado do jogo é atualizado e as condições de término do jogo são verificadas;
3. o detetive executa uma ação sobre o estado produzido pela ação do ladrão;
4. o estado do jogo é atualizado e as condições de término do jogo são verificadas;
5. o contador de turnos é decrementado e o próximo turno é iniciado.

O jogo termina com vitória do detetive se ele for capaz de capturar o ladrão. Se o ladrão conseguir roubar o tesouro escolhido e fugir da cidade onde o roubo ocorreu, o jogo termina com vitória do ladrão.

Há um número máximo de turnos definido no início do jogo pelo cenário. Se o contador de turnos atingir zero, o jogo termina com empate.

A máquina de jogo e um cenário de exemplo serão fornecidos com este enunciado. Cada agente é implementado como um módulo separado e deve ser carregado no jogo por meio de uma função apropriada.

A seguir descrevemos os módulos de cenário e dos agentes.

1.1 Arquivo de Cenário

O arquivo de cenário define:

- a lista de suspeitos;
- o mapa de cidades na forma de um grafo;
- os tesouros que podem ser objetivos do ladrão;
- itens roubáveis distribuídos pelo mapa.

O cenário do jogo é composto por cidades conectadas. O ladrão e o detetive podem se deslocar apenas entre cidades diretamente conectadas.

Por exemplo, o mapa abaixo representa as conexões entre cinco cidades denominadas **a**, **b**, **c**, **d** e **e**.

```
a ---- b
|      |
|      |
c-----d-----e
```

O grafo apresentado é descrito declarando cada cidade e, em seguida, cada conexão:

```
cidade(a).  
cidade(b).  
cidade(c).  
cidade(d).  
cidade(e).  
  
conectado(a,b).  
conectado(a,c).  
conectado(b,d).  
conectado(c,b).  
conectado(d,e).  
conectado(d,c).
```

1.1.1 Tesouros e Itens

Cada partida possui um conjunto de tesouros e itens roubáveis distribuídos pelas cidades.

O arquivo de cenário tipicamente começa definindo os seguintes predicados:

```
:- dynamic item/3.  
:- dynamic tesouro/3.  
:- dynamic roubado/2.
```

O functor `item(N,C,XS)` define:

- um item identificado por `N`;
- localizado na cidade `C`;
- e cujos pré-requisitos são os itens da lista `Xs`.

O functor `tesouro/3` possui a mesma estrutura.

O predicado `roubado(N,C)` descreve um item identificado por `N` que foi roubado da cidade `C`.

Essas definições são dinâmicas pois a máquina de jogo remove itens da base de dados à medida que eles são roubados.

A lista de itens pode ser declarada como no exemplo a seguir:

```
tesouro(cx_joias,a,[chave]).  
item(chave,c,[cartao_cofre]).  
item(cartao_cofre,e,[]).
```

Nesse caso, o ladrão deve:

1. roubar `cartao_cofre`;
2. depois roubar `chave`;
3. finalmente roubar `cx_joias`.

1.1.2 Suspeitos

O arquivo de cenário deve definir a lista de suspeitos por meio do predicado:

`procurado(ID,NOME,AP)`

em que:

- ID é um identificador numérico único;
- NOME é um átomo representando o nome do suspeito;
- AP é uma lista de características físicas.

Não existe uma convenção fixa para os atributos físicos utilizados nessa lista. Alguns exemplos de características são:

- altura;
- gênero;
- cor dos olhos;
- cor do cabelo;
- cicatrizes;
- tatuagens;
- forma da orelha;
- forma do nariz;
- porte físico.

No arquivo de exemplo `mapa1`, temos os seguintes suspeitos:

```
procurado(0,
    'Dick Vigarista',
    aparencia([altura(180), genero(gen1), corpulento,
                cor_olhos(escuro), cor_cabelo(escuro),
                ton_pele(cor1), nariz(longo),
                marca(bochecha_esquerda)] )).

procurado(1,
    'Penelope Mao Leve',
    aparencia([altura(120), genero(gen2), magro,
                cor_olhos(amarelo), cor_cabelo(escuro),
                ton_pele(cor2), nariz(curto),
                cicatriz(sobrancelha_esquerda)] )).

procurado(2,
    'Clepto Maniaco',
    aparencia([altura(160), genero(gen1), atletico,
                cor_olhos(verde), cor_cabelo(castanho),
                ton_pele(cor2), nariz(medio),
                tatuagem(testa)] )).
```

1.1.3 Número de Turnos

O número máximo de turnos é definido pelo functor:

`max_turnos(N)`.

em que `N` é um número inteiro.

1.2 Mecânica do Jogo

No início do jogo, a máquina de jogo invoca uma função especial de inicialização denominada:

`detetive_preload(G,LS,LI,LT,Ready)`

ou

`ladrao_preload(G,LS,LI,LT,Ready,ID,Obj)`

no caso do ladrão.

Os primeiros cinco parâmetros possuem o seguinte significado:

- `G`: o grafo de cidades na forma de uma lista de funtores `adj(X,Y)`;
- `LS`: a lista de suspeitos;
- `LI`: a lista de itens;
- `LT`: a lista de tesouros;
- `Ready`: deve ser instanciado com o valor `pronto` quando o agente terminar sua pré-inicialização.

No caso do ladrão, espera-se ainda:

- `ID`: o identificador do suspeito escolhido;
- `Obj`: o tesouro escolhido como alvo.

O objetivo dessas funções é apresentar aos agentes os dados públicos do cenário. Idealmente, os agentes devem copiar essas informações para suas próprias bases de conhecimento para consultas futuras.

Em seguida, a máquina de jogo invoca:

`ladrao_action(E,St,A)`

em que:

- `E` é a lista de eventos conhecidos até o momento;
- `St` é o estado do ladrão;
- `A` é a ação escolhida.

A lista de eventos contém todos os eventos observados desde o início da partida.

Espera-se que o predicado instancie a variável `A` com uma ação válida. Caso o predicado não produza nenhuma solução, a máquina irá instanciar `A` com a ação `nada`.

O estado do ladrão é representado pelo functor:

`thief(loc(C), ID, AP, OBJ, Itens, Dsg)`

em que:

- `C` é a cidade onde o ladrão se encontra;
- `ID` é o identificador do suspeito impersonado pelo ladrão;
- `AP` é a aparência atual do ladrão;
- `OBJ` é o tesouro escolhido como alvo;
- `Itens` é a lista de itens roubados;
- `Dsg` é o número de disfarces restantes.

A ação escolhida será validada antes da execução. Por exemplo, uma ação `move(A,B)` somente é válida se existir conexão entre as cidades `A` e `B`.

Caso a ação falhe na validação, ela será considerada inválida e a máquina seguirá para a vez do detetive.

Após a execução da ação do ladrão, o estado global do jogo é atualizado e a máquina invoca:

`detetive_action(E, St, A)`

O estado do detetive é representado pelo functor:

`detective(loc(C), M, CS)`

em que:

- `C` é a cidade onde o detetive se encontra;
- `M` é o mandato atual do detetive;
- `CS` é a lista de pistas coletadas.

O termo `M` será:

- `nenhum`, caso não exista mandato ativo;
- `mandato(ID)`, caso exista um mandato contra o suspeito `ID`.

A lista `CS` contém pistas observadas pelo detetive, compostas por características da aparência do ladrão. Assim como ocorre com o ladrão, a ação do detetive também será validada antes da execução.

Após a jogada do detetive, o número de turnos é decrementado e o processo é repetido.

1.2.1 Ações do Detetive

O detetive pode executar as seguintes ações:

- `move(A,B)`
Move o detetive da cidade `A` para a cidade `B`.
- `inspecionar`
Inspeciona a cidade atual. Caso o ladrão esteja presente e exista um mandato válido contra ele, o ladrão é capturado.

- **pedir_mandato(ID,XS)**
Solicita um mandato contra o suspeito ID, usando a lista de pistas XS. A lista XS deve conter apenas pistas presentes na lista de pistas do detetive.
Para que o mandato seja emitido, a lista de pistas deve reduzir a lista de suspeitos para no máximo dois suspeitos, dentre os quais um deles deve possuir o identificador ID.
- **fechar(C)**
Fecha a cidade C. O ladrão pode entrar em uma cidade fechada, mas será capturado caso tente sair dela, independentemente da existência de um mandato.
Apenas uma cidade pode permanecer fechada por vez.
- **liberar(C)**
Libera uma cidade previamente fechada.
- **nada**
Não realiza nenhuma ação.

1.2.2 Ações do Ladrão

O ladrão pode executar as seguintes ações:

- **move(A,B)**
Move o ladrão da cidade A para a cidade B.
- **roubar(I)**
Rouba o item ou tesouro I, desde que todos os pré-requisitos necessários tenham sido satisfeitos.
- **disfarce([X1,X2,...])**
Aplica modificações à aparência do ladrão.
Cada modificação consome uma unidade de disfarce e o número de modificações não pode exceder o número de disfarces disponíveis.
- **despir_disfarce**
Remove todos os disfarces ativos.
- **nada**
Não realiza nenhuma ação.

1.2.3 Disfarces

O ladrão possui um número limitado de disfarces.
Um disfarce permite:

- **adicionar(X)**: adicionar uma característica falsa. A característica X não pode já pertencer à aparência atual do ladrão;
- **omitir(X)**: ocultar a característica X;
- **trocar(X,Y)**: substituir a característica X pela característica Y.

Assim, o conjunto de pistas obtidas pelo detetive pode conter informações conflitantes.

2 Dos Artefatos a Serem Entregues

Devem ser entregues o código-fonte produzido, bem como um documento que explique como utilizar a interface do jogo.

Antes de enviar o trabalho para avaliação, assegure-se de que:

1. o código compila e executa em ambiente Unix/Linux. Programas que não compilam receberão nota zero;
2. todos os arquivos-fonte possuem, em comentário no início do arquivo, o nome e a matrícula dos autores do trabalho;
3. o arquivo de documentação contém a identificação dos autores do trabalho;
4. o arquivo compactado com os artefatos está devidamente identificado com nome e matrícula.

3 Critérios de Avaliação

A avaliação será realizada mediante análise do código-fonte, entrevista e competição entre os agentes. Os seguintes fatores serão observados na avaliação do código-fonte: corretude do programa, estrutura do código, organização e legibilidade.

Critério	Valor
Código Fonte	0,5
Entrevista	1,5
Competição	0,5

Nesse quesito, serão observados:

- estruturas de dados bem planejadas;
- legibilidade do código, isto é, uso de nomes mnemônicos para variáveis, predicados e comentários úteis.

Atrasos serão penalizados por uma função exponencial em relação aos dias úteis de atraso, isto é, será reduzido do percentual da nota o valor correspondente à exponencial de base 2 do número de dias de atraso. A Tabela a seguir mostra a nota em função dos dias de atraso:

Dias de Atraso	Nota
1	$n \times 0.98$
2	$n \times 0.96$
3	$n \times 0.92$
4	$n \times 0.84$
5	$n \times 0.68$
6	$n \times 0.36$
7	0

Observe que, a partir do sétimo dia de atraso, o trabalho não será mais avaliado.

4 Entrega do Trabalho

A data de entrega do trabalho será até o dia **30 de junho de 2026**.

A entrega será realizada via Moodle, com o envio de todos os componentes em um único arquivo compactado no formato **zip** ou **rar**.